AI ASSISTED CODING

LAB TEST-3

SET-E4:

Q1:

Scenario: In the Agriculture sector, a company faces a challenge related to code refactoring.

Task: Use AI-assisted tools to solve a problem involving code refactoring in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output.

Code :

```
 ================================================================
# Project: AI-Assisted Crop Yield Prediction
# Author: [Your Name]
# ================================================================

# Importing required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear model import LinearRegression

# ----------------------------------------------------------------
# Step 1: Store agricultural data using Python data structures
# ----------------------------------------------------------------

# Simulated dataset (can be replaced by real CSV data)
data = [
    {"Crop": "Wheat", "Rainfall": 500, "Temperature": 20, "Fertilizer": 50,
"Yield": 2.5},
    {"Crop": "Wheat", "Rainfall": 550, "Temperature": 22, "Fertilizer": 60,
"Yield": 2.8},
    {"Crop": "Rice", "Rainfall": 1200, "Temperature": 28, "Fertilizer": 80,
"Yield": 4.2},
    {"Crop": "Rice", "Rainfall": 1100, "Temperature": 27, "Fertilizer": 70,
"Yield": 3.9},
    {"Crop": "Maize", "Rainfall": 800, "Temperature": 25, "Fertilizer": 65,
"Yield": 3.0},
```

```
    {"Crop": "Maize", "Rainfall": 750, "Temperature": 24, "Fertilizer": 60,
"Yield": 2.8},
    {"Crop": "Sugarcane", "Rainfall": 1000, "Temperature": 26, "Fertilizer": 100,
"Yield": 5.0},
    {"Crop": "Cotton", "Rainfall": 600, "Temperature": 30, "Fertilizer": 70,
"Yield": 2.0}
]

# Convert to DataFrame for AI model
df = pd.DataFrame(data)

# ---------------------------------------------------------------
# Step 2: Encode categorical data (Crop) numerically
# ---------------------------------------------------------------
df["Crop_Code"] = df["Crop"].astype("category").cat.codes

# Independent and dependent variables
X = df[["Crop_Code", "Rainfall", "Temperature", "Fertilizer"]]
y = df["Yield"]

# ---------------------------------------------------------------
# Step 3: Train AI model (Linear Regression)
# ---------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

# ---------------------------------------------------------------
# Step 4: Predict yield based on user input
# ---------------------------------------------------------------
print("=== AI-Assisted Crop Yield Predictor ===")
crop = input("Enter Crop (Wheat/Rice/Maize/Sugarcane/Cotton): ")
rainfall = float(input("Enter expected rainfall (in mm): "))
temperature = float(input("Enter temperature (°C): "))
fertilizer = float(input("Enter fertilizer used (kg/acre): "))

crop_code = df.loc[df["Crop"] == crop, "Crop_Code"].iloc[0]

# Predict yield
predicted_yield = model.predict([[crop_code, rainfall, temperature, fertilizer]])

print(f"\n Predicted Yield for {crop}: {predicted_yield[0]:.2f} tons/acre")

# ---------------------------------------------------------------
# Step 5: AI Assistance Explanation (for report)
# ---------------------------------------------------------------
# AI (ChatGPT or OpenAI API) was used to:
# - Suggest suitable data structure (list of dictionaries)
# - Recommend ML model (Linear Regression)
```

```
# - Optimize input handling and output formatting
# - Generate documentation and code explanation automatically
```

Output:

```
=== AI-Assisted Crop Yield Predictor ===
Enter Crop (Wheat/Rice/Maize/Sugarcane/Cotton): Rice
Enter expected rainfall (in mm): 1150
Enter temperature (°C): 27
Enter fertilizer used (kg/acre): 75

 Predicted Yield for Rice: 4.11 tons/acre
```

Explanation:

This notebook demonstrates a simple crop yield prediction model. It uses a simulated dataset of crop information, environmental factors, and yield. The code imports necessary libraries like pandas and scikit-learn. It stores the data in a pandas DataFrame. Categorical crop names are converted to numerical codes. The data is split into training and testing sets. A Linear Regression model is trained on the data. The model takes user input for crop, rainfall, temperature, and fertilizer. It predicts the crop yield based on the input. Finally, it prints the predicted yield.

Q2:

Scenario: In the Education sector, a company faces a challenge related to backend api
development.
Task: Use AI-assisted tools to solve a problem involving backend api development in this
context.
Deliverables: Submit the source code, explanation of AI assistance used, and sample output.

Code:

```
#AI-Assisted Retail Product Recommendation System
# Author: [Your Name]

products = [
    {"name": "Laptop", "category": "Electronics", "price": 60000, "popularity":
95},
    {"name": "Headphones", "category": "Electronics", "price": 2500, "popularity":
85},
```

```
    {"name": "Shoes", "category": "Fashion", "price": 3500, "popularity": 75},
    {"name": "Smartphone", "category": "Electronics", "price": 30000,
"popularity": 98},
    {"name": "T-Shirt", "category": "Fashion", "price": 1200, "popularity": 65},
    {"name": "Refrigerator", "category": "Home Appliances", "price": 25000,
"popularity": 90},
    {"name": "Microwave", "category": "Home Appliances", "price": 8000,
"popularity": 80},
]

# Sort products by category (simple bubble sort)
for i in range(len(products)):
    for j in range(0, len(products) - i - 1):
        if products[j]["category"] > products[j + 1]["category"]:
            products[j], products[j + 1] = products[j + 1], products[j]

# Binary search for category
def find_category(data, cat):
    return [p for p in data if p["category"].lower() == cat.lower()]

# Simple AI logic: recommend popular items within budget
def ai_recommend(items, budget):
    rec = [p for p in items if p["price"] <= budget]
    return sorted(rec, key=lambda x: x["popularity"], reverse=True)[:3] or ["No
items found"]

print("=== AI-Assisted Retail Recommendation ===")
cat = input("Enter category (Electronics/Fashion/Home Appliances): ")
budget = float(input("Enter your budget: "))

results = ai recommend(find category(products, cat), budget)
print("\n Recommendations:")
for r in results:
    if isinstance(r, dict):
        print(f"- {r['name']} | ₹{r['price']} | Popularity: {r['popularity']}")
    else:
        print(r)
```

Output:

```
=== AI-Assisted Retail Recommendation ===
Enter category (Electronics/Fashion/Home Appliances): electronics
Enter your budget: 40000

 Recommendations:
- Smartphone | ₹30000 | Popularity: 98
- Headphones | ₹2500 | Popularity: 8
```

Explanation:

This notebook demonstrates a simple retail product recommendation system. It uses a list of products with categories, prices, and popularity. Products are sorted by category and can be searched by category. A function recommends popular items within a given budget. The system takes user input for category and budget to provide recommendations.